Attention-Residual Variants of GPT-2 124M: Structure, Performance, and Why Some Collapse

Experiment log, branch `attention-experiments`

2026-10-16

Summary

Five modifications of GPT-2's attention sublayer (approach_1 to approach_5) and the original GPT-2 (*baseline*) were each trained from scratch for a Chinchilla-optimal budget: 2.5B FineWeb-Edu tokens, same data order, same hyperparameters, same seed. All of them feed the raw residual stream X into attention in some form. They differ in how the token's own state is carried to the next layer.

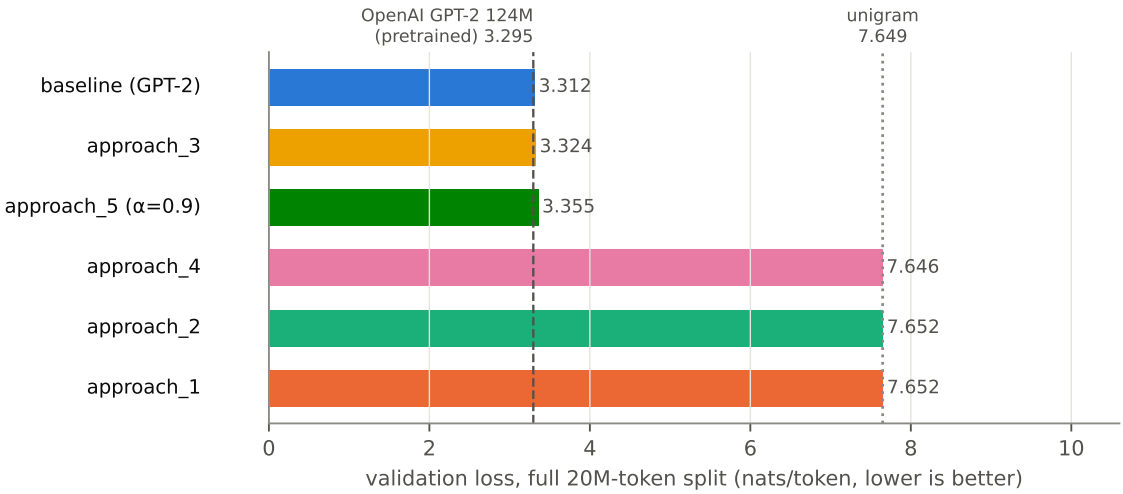


Figure 1: Final validation loss on the full 20M-token split. approach_1/2 are scored at their last checkpoint (step 3000). All the others, approach_4 included, are scored at their final checkpoint (step 4768).

Model	Skip path (carries the token's state)	Val loss (20M)	vs. baseline	Outcome
baseline	X (identity)	**3.312**	–	trains
approach_3	X (identity)	3.324	+0.011	trains, ties
approach_5	$0.9X + 0.1\,\mathrm{Concat}_h[A_hX_h]$	3.355	+0.043	trains, worse
approach_1	$\mathrm{Concat}_h[A_hX_h](W_O{+}I)$ (mixed)	7.652	–	collapsed
approach_2	$\mathrm{LN}(\cdot)(W_O{+}I)$ (mixed)	7.652	–	collapsed
approach_4	$\mathrm{Concat}_h[A_hX_h]$ (mixed)	7.646	–	collapsed
OpenAI GPT-2 124M	pretrained, reference	3.295		
Unigram model	no context, reference	7.649		

The one rule that explains every outcome. A variant trains if and only if each token's own state reaches the next layer through a path that is *not* averaged by the attention matrix. If X only travels through A_hX_h (approach_1, 2, 4), every layer replaces each token with an average over its prefix. After a few layers all positions hold the same vector, already at initialization, and the attention logits then receive exactly zero gradient. These models settle on a context-free unigram predictor. Keeping a full identity skip (approach_3) gives the baseline's quality. Keeping 90% of it (approach_5, $\alpha = 0.9$) trains but pays +0.043 nats. Adding X_h inside attention (approach_3) gives no measurable benefit.

1 Setup

Data	FineWeb-Edu `sample-10BT` (GPT-2 BPE); first 2.5B tokens for training; a disjoint 20M-token validation split. Curve points use its first 4.2M tokens.
Model	GPT-2 124M (L=12, H=12, D=768, d_h=64, T=1024, tied embeddings). No variant adds parameters (124,475,904 each; α is a fixed constant).
Training	4768 steps × 524,288 tokens; AdamW (0.9, 0.95), wd 0.1, clip 1.0; LR 6×10^{-4}, 250 warmup steps, cosine to 10%; bf16; init $\mathcal{N}(0, 0.02^2)$ with output projections scaled by $1/\sqrt{2L}$.
Hardware	baseline and approach_1/2: 1 H100 each; approach_3/4/5: 4 H100s (DDP). The DDP code splits each step's micro-batches across GPUs. A 30-step check reproduced the single-GPU baseline's loss and gradient norm to 4 decimals.
Stopped runs	approach_1 at step 3951 and approach_2 at 3741, after they reached the collapse plateau. approach_4 was first stopped at step 270, then rerun to the full 4768 steps to check that the plateau holds.
Code	`train/gpt2_torch.py` (all variants), `train/train.py`; the TF reference `src/model.py` implements the same variants via `attn_proj_residual`.

2 Architectures

2.1 Notation

Row-vector convention: $X \in \mathbb{R}^{T\times D}$ is the residual stream entering a block, and

$$[Q \,|\, K \,|\, V] = \mathrm{LN}_1(X)\, W_{QKV} + b_{QKV}, \qquad A_h = \mathrm{softmax}\big(Q_h K_h^\top / \sqrt{d_h} + M\big),$$

with Q, K, V split into heads $Q_h, K_h, V_h \in \mathbb{R}^{T\times d_h}$ and A_h the causal, row-stochastic attention matrix. The quantity all approaches share is the head slice of the *raw* residual,

$$X_h = X[:,\, hd_h : (h{+}1)d_h].$$

$W_O \in \mathbb{R}^{D\times D}$ is the output projection. Every block ends with the standard MLP sublayer $X^{(\ell+1)} = X' + \mathrm{MLP}(\mathrm{LN}_2(X'))$, so the variants differ only in X'.

2.2 The six attention sublayers

$$\begin{aligned}
\textbf{baseline}: &\quad X' = X + \mathrm{Concat}_h[A_hV_h]\, W_O + b_O \\
\textbf{approach_1}: &\quad Y = \mathrm{Concat}_h[A_h(V_h + X_h)], \qquad X' = YW_O + b_O + Y \\
\textbf{approach_2}: &\quad \hat{Y} = \mathrm{LN}\big(\mathrm{Concat}_h[A_h(V_h + X_h)]\big), \qquad X' = \hat{Y}W_O + b_O + \hat{Y} \\
\textbf{approach_3}: &\quad X' = X + \mathrm{Concat}_h[A_h(V_h + X_h)]\, W_O + b_O \\
\textbf{approach_4}: &\quad X' = \mathrm{Concat}_h[A_hV_h]\, W_O + b_O + \mathrm{Concat}_h[A_hX_h] \\
\textbf{approach_5}: &\quad X' = \mathrm{Concat}_h[A_hV_h]\, W_O + b_O + \alpha X + (1-\alpha)\,\mathrm{Concat}_h[A_hX_h], \qquad \alpha = 0.9
\end{aligned}$$

The LN in approach_2 has no learned parameters. In approach_4 and 5, $A_h V_h$ and $A_h X_h$ use the same attention matrix A_h.

2.3 A common form: attention term plus skip path

Every variant can be written as $X' = \underbrace{(\text{attention term})}_{\text{reads context}} + \underbrace{S(X)}_{\text{skip path}}$, where the skip path is what carries the token's own state. Expanding the concatenation over heads ($W_O^{(h)}$ are the rows of W_O for head h):

Variant	Skip path $S(X)$	Row i of $S(X)$ depends on
baseline, approach_3	X	x_i only
approach_5	$\alpha X + (1-\alpha)\,\mathrm{Concat}_h[A_h X_h]$	$\alpha\, x_i \;+\; (1-\alpha)\times$ average of $x_{\le i}$
approach_4	$\mathrm{Concat}_h[A_h X_h]$	average of $x_{\le i}$
approach_1	$\mathrm{Concat}_h[A_h X_h] + \sum_h A_h X_h W_O^{(h)}$	average of $x_{\le i}$, then projected
approach_2	part of $\mathrm{LN}(\mathrm{Concat}_h[A_h(V_h{+}X_h)])(W_O{+}I)$	average of $x_{\le i}$, normalized

In approach_3 the new X_h term sits inside the attention term, as $\sum_h A_h X_h W_O^{(h)}$, and the identity skip is untouched. In approach_1, 2 and 4, X reaches the next layer *only* through A_h. approach_5 interpolates between the two with α.

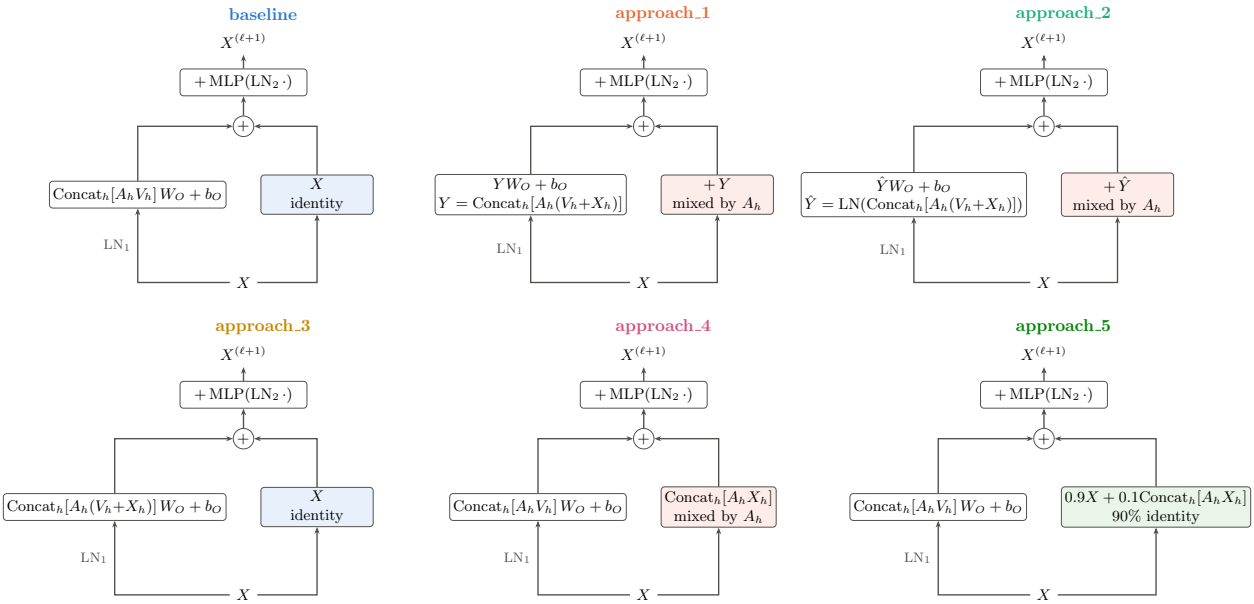


Figure 2: Each variant as an attention term (left) plus a skip path (right) that carries the token's own state. Skip-path fill: blue, an identity path (x_i passes unchanged); red, the state reaches the next layer only after averaging by A_h; green, a 90/10 blend. In approach_1 and 2, Y also feeds the attention term, so the box shows only the part that bypasses W_O. Red skip paths collapsed in training; blue and green trained.

3 Performance

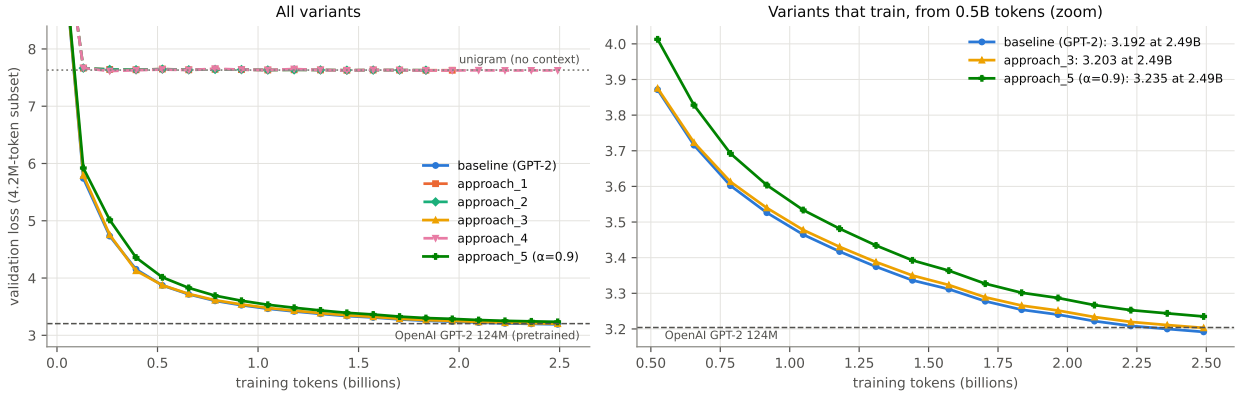


Figure 3: Validation loss (4.2M-token subset) every 250 steps. Left: every variant either follows the baseline or drops to the unigram line within 250 steps and stays there. Right: zoom on the three that train. The dashed line is the pretrained GPT-2 on the same 4.2M tokens (3.204).

	Val loss at step (4.2M subset)					Final
Model	250	500	1000	2000	3000	20M split
baseline	5.741	4.733	3.871	3.465	3.312	3.312
approach_3	5.793	4.755	3.875	3.478	3.323	3.324
approach_5	5.921	5.015	4.013	3.534	3.364	3.355
approach_1	7.665	7.642	7.649	7.632	7.632	7.652
approach_2	7.665	7.642	7.647	7.632	7.631	7.652
approach_4	7.665	7.614	7.640	7.631	7.633	7.646

approach_4 over the full run. The first approach_4 run was stopped at step 270. It was rerun for all 4768 steps to check that the collapse is permanent. From step 250 to the end, its validation loss stays between 7.61 and 7.67 with no downward trend. Its final loss on the 20M split, 7.646, equals the unigram model's 7.649. Its gradients differ from approach_1/2's. After step 400 the pre-clip gradient norm is large and erratic: median 2.1, 90th percentile 15 (approach_1: 0.14; baseline: 0.3; Figure 5). Clipping caps each update at norm 1.0, and the loss does not move in response.

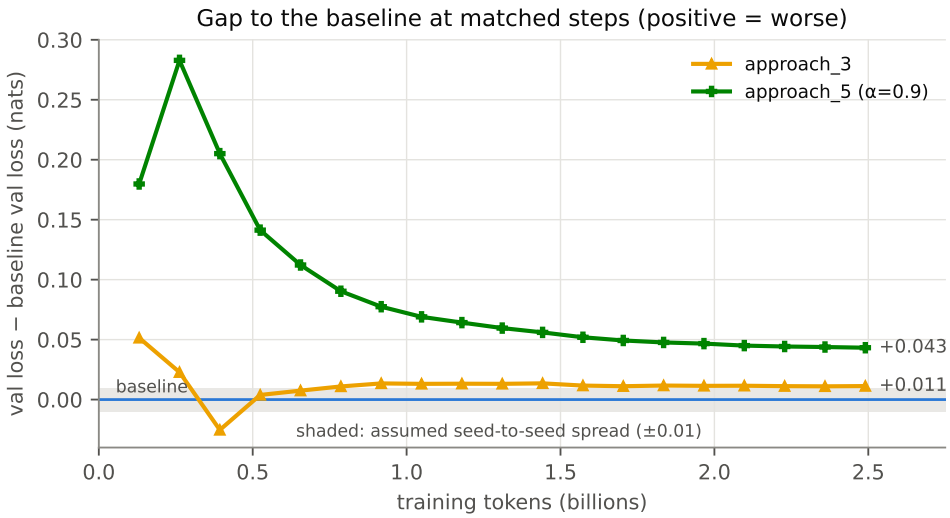


Figure 4: Validation-loss gap to the baseline at the same step. approach_3 settles at about +0.011 from 1B tokens on, which is about the size of seed-to-seed variation. approach_5 starts far behind (peak +0.28 at step 500), closes quickly until about 1B tokens, then flattens at +0.043.

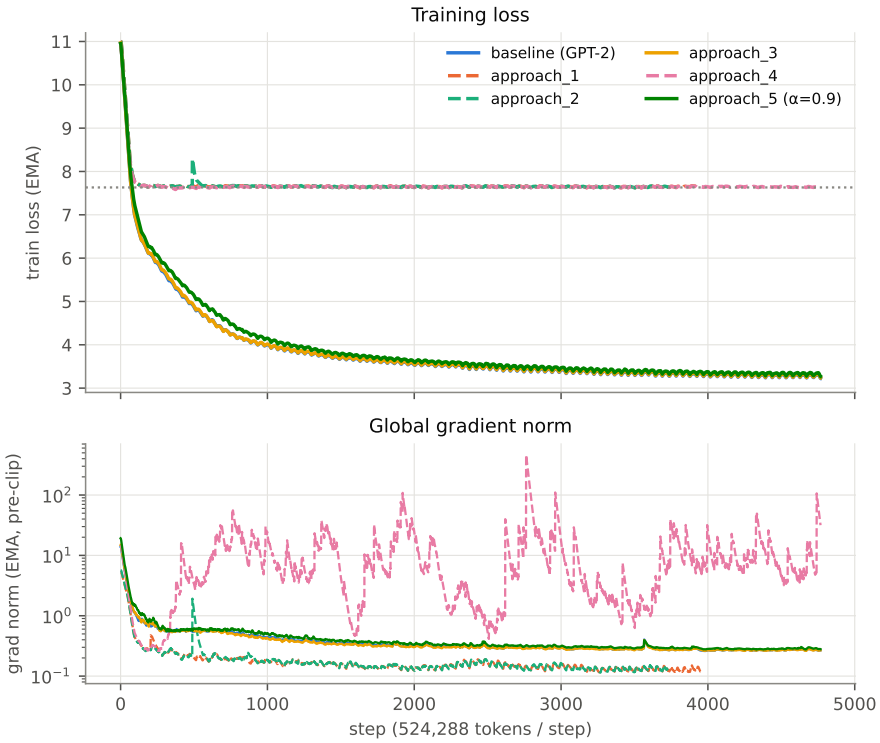


Figure 5: Training loss and gradient norm. The collapsed variants (dashed) reach the unigram plateau by step ∼150, and their gradient norm settles 2–3× below the trained models'. approach_2's spike at step 489 (loss 20.3) returns to the same plateau within about 10 steps.

Reading the results.

- **Three variants collapse.** approach_1, 2 and 4 match the unigram model to within 0.004 nats (7.652, 7.652 vs. 7.649) and their predictions are identical at every position (Section 4.1).
- **approach_3 ties the baseline.** Its gap of +0.011 is consistent across 14 checkpoints but is about the size of seed noise for runs this size (only one seed per variant was run).
- **approach_5 is clearly worse** by +0.043, about 4× that noise level. The gap shrinks by only ∼0.001 per 0.13B tokens at the end, so it would not close within this budget.
- The baseline ends level with OpenAI's pretrained GPT-2 (3.312 vs. 3.295), which confirms the training pipeline. GPT-2 is scored off its training distribution, so this is context only.

4 Why some variants work and others do not

4.1 Averaging the skip path causes rank collapse, already at initialization

At initialization Q, K are small, so $A_h \approx P$ with $P_{ij} = \frac{1}{i+1}[j \le i]$. If the skip path is $\text{Concat}_h[A_h X_h] \approx PX$, every layer replaces each token's state with the mean of its prefix. P is row-stochastic and its only fixed direction is the constant vector, so repeated application drives every position toward the same vector: the residual stream becomes rank 1. approach_5 applies $\alpha I + (1-\alpha)P$ instead, which keeps a fraction α of each token's own state per layer. The attention and MLP outputs add further (partly averaged) content on top in every variant.

We measured this directly on the randomly initialized 12-layer models. *Distance from rank 1* is $\|X - \mathbf{1}\bar{x}^\top\|_F / \|X\|_F$: 1 for unrelated positions, 0 when every position holds the same vector.

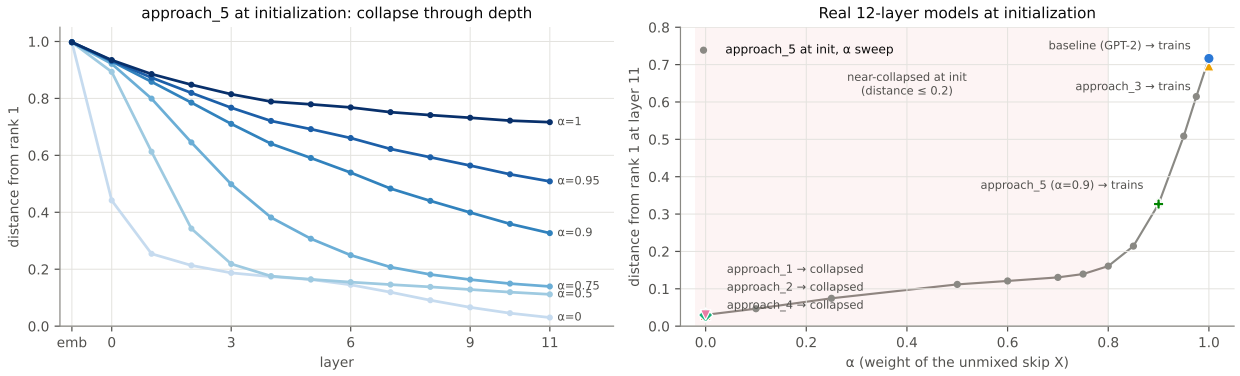


Figure 6: Left: distance from rank 1 through depth for approach_5 at initialization, for several α (α=0 is approach_4; α=1 is the baseline's identity skip plus approach_4's attention term). Right: the layer-11 value against α, with every variant's initialization marked and its training outcome annotated. Every variant at or below about 0.15 collapsed in training; every variant at 0.33 or above trained.

Model	At initialization (layer 11) distance from rank 1	cosine between positions	Trained (layer 11) distance from rank 1	Outcome
baseline	0.717	0.487	0.877	trains
approach_3	0.696	0.515	0.878	trains
approach_5	0.327	0.894	0.872	trains (recovers)
approach_1	0.029	0.999	0.002	collapsed
approach_2	0.030	0.999	0.000	collapsed
approach_4	0.030	0.999	0.001	collapsed

This initialization probe takes seconds and correctly predicted the outcome of all six runs before training. approach_4's collapse was predicted from it and then confirmed over the full 2.5B tokens.

4.2 Once collapsed, attention cannot learn: a zero-gradient fixed point

Proposition 1. *If a layer's input is identical at every position, $X = \mathbf{1}x^\top$, then that layer's attention is uniform and the loss gradient with respect to its attention logits is exactly zero.*

Proof. LN_1 is per-position, so all q_i, k_j, v_j and $x_{h,j}$ are equal. The logits $S_{ij} = q \cdot k/\sqrt{d_h}$ are constant over the causal prefix, so A is uniform. With $u_j = v_j + x_{h,j} = u$ (or $u_j = v_j$, or $u_j = x_{h,j}$) and $g_i = \partial\mathcal{L}/\partial O_i$, the softmax Jacobian gives $\partial\mathcal{L}/\partial S_{ij} = A_{ij}\big(g_i \cdot u_j - \sum_k A_{ik}\, g_i \cdot u_k\big) = 0$. □

At initialization, the collapsed variants' streams are within 3% of rank 1 by layer 11 and within 26% after layer 1, so the upper layers get almost no signal to sharpen their attention. Fitting token frequencies needs only a constant output, so the optimizer finds it easier to collapse *further*: the layer-11 distance falls from 0.03 to $\leq$0.002, and the per-position KL to the unigram distribution is 0.005, 0.004 and 0.007 (approach_1, 2, 4). In approach_1/2, the attention in every layer above 0 is exactly uniform (Figures 7 and 8).

approach_4 reaches the same end state by a different route. Its trained attention is *sharp*, not uniform: in layers 0–2 the largest weight per query averages 0.85–0.94. But within a head, most queries choose the same source position: 28%, 47% and 83% of queries in layers 0, 1 and 2, measured on 4 validation sequences, positions ≥ 64. In layers 5 and 11 every query attends to the same few early tokens (the vertical stripes in Figure 8). Since its skip path is $\mathrm{Concat}_h[A_h X_h]$, every token then copies the same X_h. The positions become identical (cosine 1.000 from layer 4 on; layer-11 distance from rank 1 0.001), and the output is again the unigram distribution. Sharp attention to a *shared* source collapses the stream just as uniform attention does. Only an unmixed path for each token's own state prevents it.

approach_5 starts partly collapsed (0.33), but not at the fixed point. Its gradients are small but nonzero, so the heads can sharpen. They do: by the end of training the stream's distance from rank 1 is 0.872, the same as the baseline's 0.877. The slow start is visible as the gap's peak of +0.28 at step 500 (Figure 4).

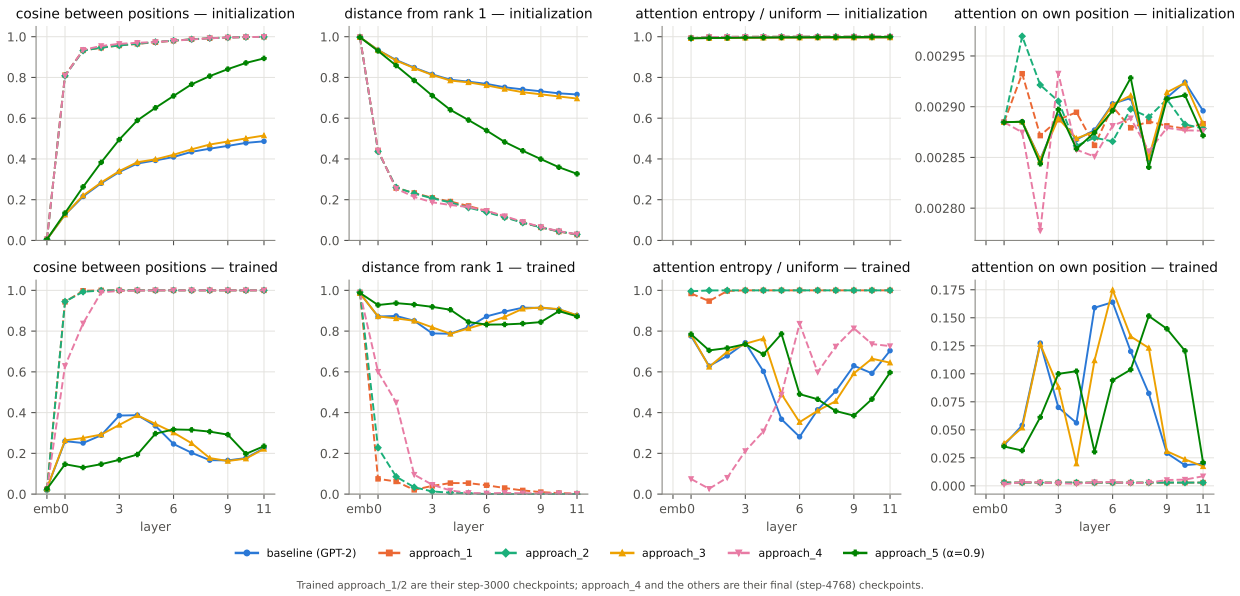


Figure 7: Per-layer diagnostics on 8 validation sequences (positions ≥ 64). Top row: initialization. Bottom row: trained. All three collapsed variants (dashed) end with identical positions (cosine 1) and near-zero self-weight. approach_1/2 do so with uniform attention (entropy ratio 1); approach_4 does so with sharper attention (entropy ratio 0.03–0.8) aimed at shared positions. approach_5 starts between the two groups and ends with the trained ones.

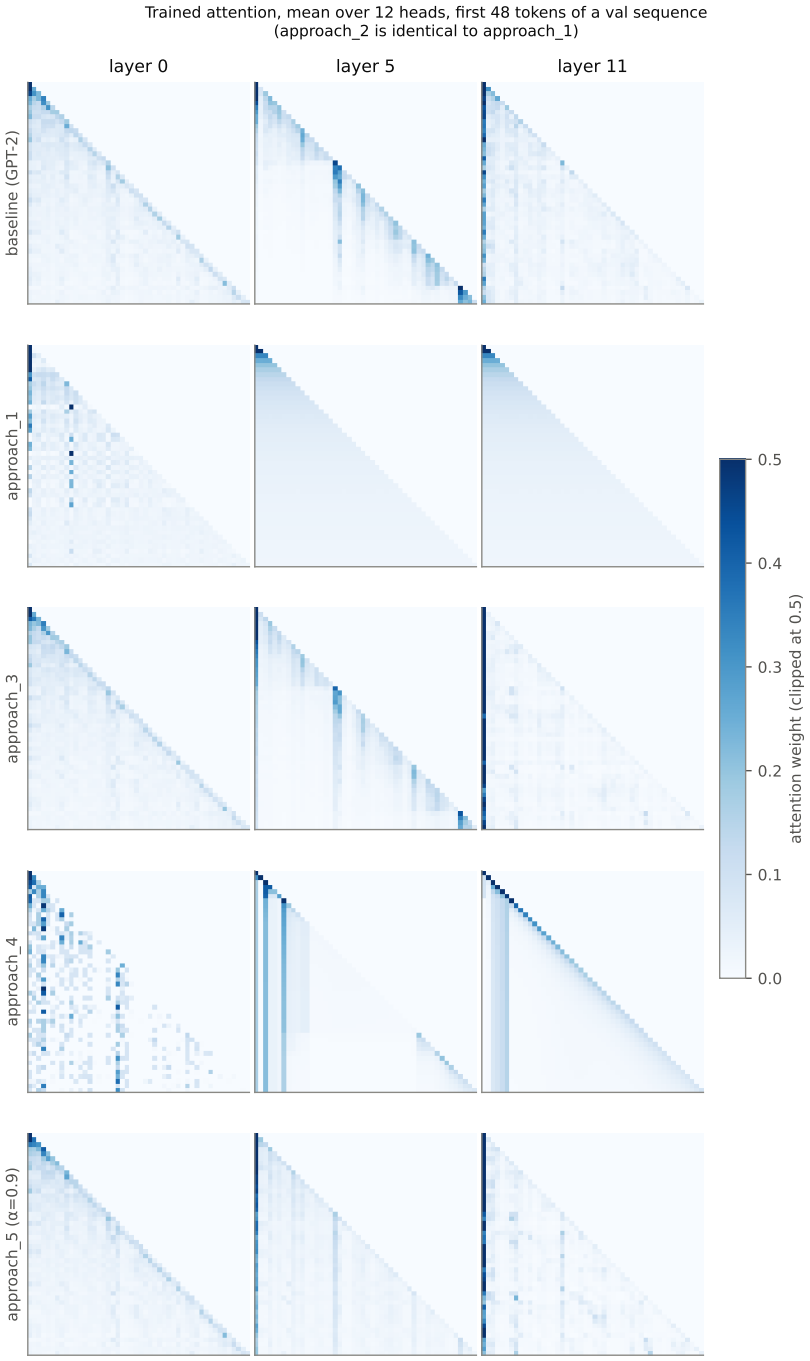


Figure 8: Trained attention, mean over heads. approach_1's layers 5 and 11 are exactly uniform over the prefix (row i is $1/(i{+}1)$). The baseline and approach_3 have nearly identical patterns. approach_5 has less of the baseline's local and diagonal structure at layer 5, and more attention on the first token. approach_4 (full run) is sharp but shared: in layers 5 and 11, every query attends to the same few early positions (vertical stripes).

4.3 Why approach_5 trains but costs +0.043 nats

Three properties of $\alpha X + (1-\alpha)\mathrm{Concat}_h[A_h X_h]$ make it strictly harder than the identity skip:

1. **Earlier layers fade.** The skip path scales the stream by α per layer, so whatever layer ℓ writes reaches the top attenuated by $\alpha^{11-\ell}$: $0.9^{11} \approx 0.31$ for layer 0. The baseline keeps it at full strength. In the trained model, the residual norm stays near 22 from layer 5 to layer 10, while the baseline's grows to 37 (Figure 9, right). Information that must survive to the top has to be re-written by later layers.

2. **One attention matrix, two jobs.** A_h both reads context (through V_h) and decides what is blended into the token's own state (through X_h). A head that attends far away for V_h also blurs X_h by 10%. The trained approach_5 moves self-attention to layers 8–10 (weight 0.12–0.15 vs. 0.02–0.08 in the baseline, Figure 9, middle). This is consistent with heads limiting the blur, but the summed self-weight over all layers is similar (0.99 vs. 0.94), so it is a reallocation, not a general increase.

3. **Slow start.** At initialization the stream is already partly collapsed (0.33 vs. 0.72), so the first ∼500 steps make little use of context (Figure 4).

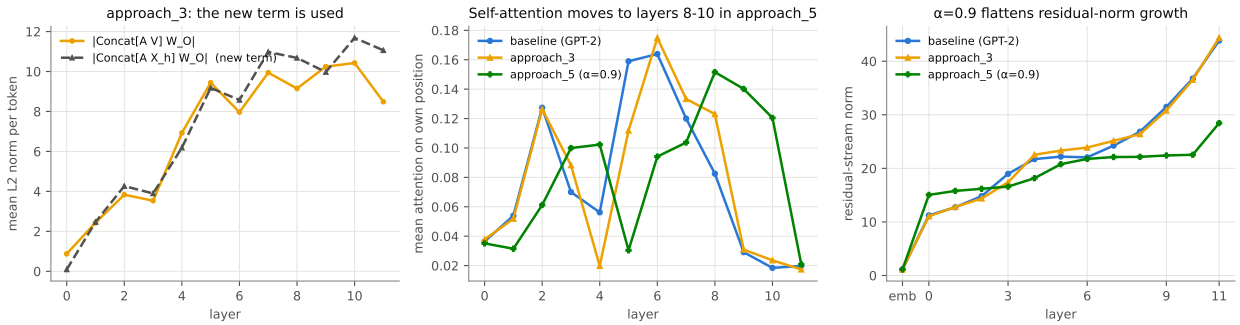


Figure 9: Trained models. Left: in approach_3, the new term $\text{Concat}_h[A_h X_h]W_O$ is as large as the standard attention term in most layers. Middle: per-layer self-attention weight. Right: residual-stream norm through depth; with $\alpha = 0.9$ it stays near 22 from layer 5 to 10.

4.4 Why approach_3 ties the baseline but does not beat it

approach_3 keeps the identity skip, so it does not collapse. Its new term is used heavily. In most layers the norm of $\text{Concat}_h[A_h X_h]W_O$ is about that of the standard term $\text{Concat}_h[A_h V_h]W_O$ (Figure 9, left). But it adds little the model could not already express. For head h,

$$A_h(V_h + X_h)\, W_O^{(h)} = A_h\big(\text{LN}_1(X)\, W_V^{(h)} + b_V^{(h)} + X E_h\big)\, W_O^{(h)},$$

where E_h selects the head's 64 coordinates. Both V_h and X_h are linear maps of the token state. They differ only in that V_h reads the per-token normalized state through a learned $W_V^{(h)}$, while X_h reads the unnormalized state through a fixed coordinate slice. The baseline can already learn $W_V^{(h)} \approx E_h$ up to LayerNorm's per-token scale. So approach_3 mainly changes the parametrization (a fixed, unnormalized extra channel) rather than adding capability, which is consistent with the near-identical loss and attention patterns.

5 Conclusions

1. **Keep an unmixed path for each token's own state.** Every variant without one (approach_1, 2, 4) collapsed to the unigram model, at initialization and permanently. Normalizing

(approach_2) or moving W_O (approach_4) does not change this.

2. **Partial mixing trains but costs quality.** approach_5 with $\alpha = 0.9$ recovers from a partly collapsed start but ends +0.043 nats behind, mainly because the skip path is no longer an identity.
3. **Feeding X_h into attention is harmless but not helpful** (approach_3, +0.011, within noise). V_h is already a learned linear read of the same state.
4. **The initialization probe is a reliable, nearly free screen.** A layer-11 distance from rank 1 at or below about 0.15 predicted collapse for every variant tested.

Suggested next steps. (i) Make α a learnable per-layer scalar initialized at 1. If training keeps it near 1, that confirms mixing the skip path does not help. (ii) Run 2–3 seeds of baseline and approach_3 to establish whether +0.011 is real (∼35 min per run on 4 GPUs). (iii) Any new variant: run the initialization probe first.

Files

`report.tex`, `report.pdf`, `numbers.tex`	this report; all numbers are generated from the logs
`figures/`	all figures (PDF and PNG)
`data/<model>/log.jsonl`, `config.json`	per-step train loss, LR, grad norm, timing; val loss every 250 steps
`data/matched_evals.json`	every model, the pretrained GPT-2 and the unigram on identical 4.2M and 20M val subsets
`data/variants_probe.json`	per-layer diagnostics, initialization and trained
`data/alpha_sweep.json`	approach_5 initialization probe over α
`data/heatmaps.npz`, `data/summary.json`	attention maps; final numbers
`code/model.py`, `code/gpt2_torch.py`	snapshot of `src/model.py` (code in Appendix A) and the trained PyTorch port

Regenerate with `train/analyze_variants.py` (GPU diagnostics and matched evals) and `train/make_variant_report.py` (figures and numbers), then `pdflatex report.tex`. Checkpoints are in `runs/chinchilla/` under `/local/scratch/a/liu3170`.

A Code for each variant (from `src/model.py`)

All listings are taken verbatim from `src/model.py`, including comments. A snapshot of the file is saved in `code/model.py`. Line numbers are the file's own. For each variant, only the lines of `attn` and `block` that run for its `hparams` setting are shown. A jump in the line numbers means the lines in between belong to other variants or to the `past` (incremental decoding) path. The training runs used the PyTorch port `train/gpt2_torch.py` (snapshot in `code/gpt2_torch.py`), which implements the same equations.

A.1 Shared code

The settings (`default_hparams`), the LayerNorm with the `affine` switch that approach_2 uses, and the per-head attention `multihead_attn`. Its `x_heads` argument adds X_h to V_h before A_h is applied.

`default_hparams()`

```
18 def default_hparams():
19     return HParams(
20         n_vocab=0,
21         n_ctx=1024,
22         n_embd=768,
23         n_head=12,
24         n_layer=12,
25         attn_residual=True,  # attention reads A_h (V_h + X_h) and replaces the skip path x = x + a
26         # Y = Concat_h[A_h (V_h + X_h)]. None: Y W_O + b_O; 'approach_1': Y W_O + b_O + Y;
27         # 'approach_2': Y <- LN(Y) (no affine), then Y W_O + b_O + Y
28         # 'approach_3': Y W_O + b_O, and block keeps the identity skip x = x + a
29         # 'approach_4': Concat_h[A_h V_h] W_O + b_O + Concat_h[A_h X_h] (V_h and X_h attended separately)
30         # 'approach_5': Concat_h[A_h V_h] W_O + b_O + attn_alpha X + (1 - attn_alpha) Concat_h[A_h X_h]
31         attn_alpha=0.9,
32         attn_proj_residual='approach_2',
33     )
```

`norm()`

```
49 def norm(x, scope, *, axis=-1, epsilon=1e-5, affine=True):
50     """Normalize to mean = 0, std = 1, then do a diagonal affine transform."""
51     with tf.variable_scope(scope):
52         u = tf.reduce_mean(x, axis=axis, keepdims=True)  # [B, T, 1]
53         s = tf.reduce_mean(tf.square(x-u), axis=axis, keepdims=True)  # [B, T, 1]
54         x = (x - u) * tf.rsqrt(s + epsilon)  # [B, T, D]
55         if not affine:
56             return x  # no g, b variables, so pretrained checkpoints still restore
57         n_state = x.shape[-1].value
58         g = tf.get_variable('g', [n_state], initializer=tf.constant_initializer(1))  # [D]
59         b = tf.get_variable('b', [n_state], initializer=tf.constant_initializer(0))  # [D]
60         x = x*g + b  # [B, T, D]
61         return x
```

`multihead_attn()`, inside `attn()`

```
114     def multihead_attn(q, k, v, x_heads=None):
115         # q, k, v, x_heads have shape [batch, heads, sequence, features]
116         w = tf.matmul(q, k, transpose_b=True)  # [B, H, T, D/H] @ [B, H, D/H, S] -> [B, H, T, S]
117         w = w * tf.rsqrt(tf.cast(v.shape[-1].value, w.dtype))  # [B, H, T, S]
118
119         w = mask_attn_weights(w)  # [B, H, T, S]
120         logits = w                # pre-softmax attention scores (scaled, masked); [B, H, T, S]
121         w = softmax(w)             # post-softmax attention weights (what the model actually uses); [B,
    ↪ H, T, S]
122         if x_heads is not None:
```

```
            v = v + x_heads  # V_h + X_h: [B, H, T, D/H]
        a = tf.matmul(w, v)  # [B, H, T, S] @ [B, H, S, D/H] -> [B, H, T, D/H]
        return a, w, logits
```

A.2 Baseline (original GPT-2)

$X' = X + \mathrm{Concat}_h[A_h V_h]\, W_O + b_O.$

`attn_residual=False, attn_proj_residual=None`

in `attn()`

```
        c = conv1d(x, 'c_attn', n_state*3)  # [B, T, D] -> [B, T, 3D]
        q, k, v = map(split_heads, tf.split(c, 3, axis=2))  # three [B, T, D], each split to [B, H, T,
    ↪ D/H]
        present = tf.stack([k, v], axis=1)  # [B, 2, H, T, D/H]
        if hparams.attn_residual:
        else:
            x_heads = None
        a, w, logits = multihead_attn(q, k, v, x_heads)  # a: [B, H, T, D/H]; w, logits: [B, H, T, S]
        a = merge_heads(a)  # [B, T, D]
        y = a  # Y = Concat_h[A_h (V_h + X_h)], pre-projection; [B, T, D]
        a = conv1d(a, 'c_proj', n_state)  # [B, T, D]
        return a, present, w, logits
```

in `block()`

```
        a, present, w, logits = attn(norm(x, 'ln_1'), 'attn', nx, past=past, hparams=hparams,
    ↪ residual_x=x)  # norm: [B, T, D]; a: [B, T, D]; w, logits: [B, H, T, S]
        if hparams.attn_residual and hparams.attn_proj_residual != 'approach_3':
        else:
            x = x + a  # [B, T, D]
        m = mlp(norm(x, 'ln_2'), 'mlp', nx*4, hparams=hparams)  # norm: [B, T, D]; m: [B, T, D]
        x = x + m  # [B, T, D]
        return x, present, w, logits
```

A.3 approach_1

$Y = \mathrm{Concat}_h[A_h(V_h + X_h)], \quad X' = YW_O + b_O + Y.$

`attn_residual=True, attn_proj_residual='approach_1'`

in `attn()`

```
        c = conv1d(x, 'c_attn', n_state*3)  # [B, T, D] -> [B, T, 3D]
        q, k, v = map(split_heads, tf.split(c, 3, axis=2))  # three [B, T, D], each split to [B, H, T,
    ↪ D/H]
        present = tf.stack([k, v], axis=1)  # [B, 2, H, T, D/H]
        if hparams.attn_residual:
            x_heads = split_heads(residual_x)  # [B, T, D] -> [B, H, T, D/H]
        a, w, logits = multihead_attn(q, k, v, x_heads)  # a: [B, H, T, D/H]; w, logits: [B, H, T, S]
        a = merge_heads(a)  # [B, T, D]
        y = a  # Y = Concat_h[A_h (V_h + X_h)], pre-projection; [B, T, D]
        a = conv1d(a, 'c_proj', n_state)  # [B, T, D]
        if hparams.attn_proj_residual in ('approach_1', 'approach_2'):
            a = a + y  # Y W_O + b_O + Y; [B, T, D]
        return a, present, w, logits
```

in `block()`

```
        a, present, w, logits = attn(norm(x, 'ln_1'), 'attn', nx, past=past, hparams=hparams,
    ↪ residual_x=x)  # norm: [B, T, D]; a: [B, T, D]; w, logits: [B, H, T, S]
        if hparams.attn_residual and hparams.attn_proj_residual != 'approach_3':
            x = a  # a = Concat_h[A_h (V_h + X_h)] W_O already carries x; [B, T, D]
```

```
178         m = mlp(norm(x, 'ln_2'), 'mlp', nx*4, hparams=hparams)  # norm: [B, T, D]; m: [B, T, D]
179         x = x + m  # [B, T, D]
180         return x, present, w, logits
```

A.4 approach_2

$Y = \mathrm{LN}(\mathrm{Concat}_h[A_h(V_h + X_h)])$ (no affine), $X' = YW_O + b_O + Y$.
`attn_residual=True, attn_proj_residual='approach_2'`

in `attn()`

```
128         c = conv1d(x, 'c_attn', n_state*3)  # [B, T, D] -> [B, T, 3D]
129         q, k, v = map(split_heads, tf.split(c, 3, axis=2))  # three [B, T, D], each split to [B, H, T,
        ↪ D/H]
130         present = tf.stack([k, v], axis=1)  # [B, 2, H, T, D/H]
135         if hparams.attn_residual:
139             x_heads = split_heads(residual_x)  # [B, T, D] -> [B, H, T, D/H]
151         a, w, logits = multihead_attn(q, k, v, x_heads)  # a: [B, H, T, D/H]; w, logits: [B, H, T, S]
152         a = merge_heads(a)  # [B, T, D]
153         if hparams.attn_proj_residual == 'approach_2':
154             a = norm(a, 'ln_y', affine=False)  # Y <- LN(Y); [B, T, D]
155         y = a  # Y = Concat_h[A_h (V_h + X_h)], pre-projection; [B, T, D]
156         a = conv1d(a, 'c_proj', n_state)  # [B, T, D]
157         if hparams.attn_proj_residual in ('approach_1', 'approach_2'):
158             a = a + y  # Y W_O + b_O + Y; [B, T, D]
159         return a, present, w, logits
```

in `block()`

```
173         a, present, w, logits = attn(norm(x, 'ln_1'), 'attn', nx, past=past, hparams=hparams,
        ↪ residual_x=x)  # norm: [B, T, D]; a: [B, T, D]; w, logits: [B, H, T, S]
174         if hparams.attn_residual and hparams.attn_proj_residual != 'approach_3':
175             x = a  # a = Concat_h[A_h (V_h + X_h)] W_O already carries x; [B, T, D]
178         m = mlp(norm(x, 'ln_2'), 'mlp', nx*4, hparams=hparams)  # norm: [B, T, D]; m: [B, T, D]
179         x = x + m  # [B, T, D]
180         return x, present, w, logits
```

A.5 approach_3

$Y = \mathrm{Concat}_h[A_h(V_h + X_h)]$, $X' = X + YW_O + b_O$.
`attn_residual=True, attn_proj_residual='approach_3'`

in `attn()`

```
128         c = conv1d(x, 'c_attn', n_state*3)  # [B, T, D] -> [B, T, 3D]
129         q, k, v = map(split_heads, tf.split(c, 3, axis=2))  # three [B, T, D], each split to [B, H, T,
        ↪ D/H]
130         present = tf.stack([k, v], axis=1)  # [B, 2, H, T, D/H]
135         if hparams.attn_residual:
139             x_heads = split_heads(residual_x)  # [B, T, D] -> [B, H, T, D/H]
151         a, w, logits = multihead_attn(q, k, v, x_heads)  # a: [B, H, T, D/H]; w, logits: [B, H, T, S]
152         a = merge_heads(a)  # [B, T, D]
155         y = a  # Y = Concat_h[A_h (V_h + X_h)], pre-projection; [B, T, D]
156         a = conv1d(a, 'c_proj', n_state)  # [B, T, D]
159         return a, present, w, logits
```

in `block()`

```
173         a, present, w, logits = attn(norm(x, 'ln_1'), 'attn', nx, past=past, hparams=hparams,
        ↪ residual_x=x)  # norm: [B, T, D]; a: [B, T, D]; w, logits: [B, H, T, S]
174         if hparams.attn_residual and hparams.attn_proj_residual != 'approach_3':
176         else:
```

```
            x = x + a  # [B, T, D]
        m = mlp(norm(x, 'ln_2'), 'mlp', nx*4, hparams=hparams)  # norm: [B, T, D]; m: [B, T, D]
        x = x + m  # [B, T, D]
        return x, present, w, logits
```

A.6 approach_4

$X' = \mathrm{Concat}_h[A_h V_h]\, W_O + b_O + \mathrm{Concat}_h[A_h X_h]$.

`attn_residual=True, attn_proj_residual='approach_4'`

in `attn()`

```
        c = conv1d(x, 'c_attn', n_state*3)  # [B, T, D] -> [B, T, 3D]
        q, k, v = map(split_heads, tf.split(c, 3, axis=2))  # three [B, T, D], each split to [B, H, T,
    ↪ D/H]
        present = tf.stack([k, v], axis=1)  # [B, 2, H, T, D/H]
        if hparams.attn_residual:
            x_heads = split_heads(residual_x)  # [B, T, D] -> [B, H, T, D/H]
        if hparams.attn_proj_residual in ('approach_4', 'approach_5'):
            a, w, logits = multihead_attn(q, k, v)  # A_h V_h: [B, H, T, D/H]
            ax = merge_heads(tf.matmul(w, x_heads))  # Concat_h[A_h X_h]: [B, H, T, S] @ [B, H, S, D/H]
    ↪ -> [B, T, D]
            a = conv1d(merge_heads(a), 'c_proj', n_state) + ax  # [B, T, D]
            return a, present, w, logits
```

in `block()`

```
        a, present, w, logits = attn(norm(x, 'ln_1'), 'attn', nx, past=past, hparams=hparams,
    ↪ residual_x=x)  # norm: [B, T, D]; a: [B, T, D]; w, logits: [B, H, T, S]
        if hparams.attn_residual and hparams.attn_proj_residual != 'approach_3':
            x = a  # a = Concat_h[A_h (V_h + X_h)] W_O already carries x; [B, T, D]
        m = mlp(norm(x, 'ln_2'), 'mlp', nx*4, hparams=hparams)  # norm: [B, T, D]; m: [B, T, D]
        x = x + m  # [B, T, D]
        return x, present, w, logits
```

A.7 approach_5

$X' = \mathrm{Concat}_h[A_h V_h]\, W_O + b_O + \alpha X + (1-\alpha)\mathrm{Concat}_h[A_h X_h]$, $\alpha = 0.9$.

`attn_residual=True, attn_proj_residual='approach_5', attn_alpha=0.9`

in `attn()`

```
        c = conv1d(x, 'c_attn', n_state*3)  # [B, T, D] -> [B, T, 3D]
        q, k, v = map(split_heads, tf.split(c, 3, axis=2))  # three [B, T, D], each split to [B, H, T,
    ↪ D/H]
        present = tf.stack([k, v], axis=1)  # [B, 2, H, T, D/H]
        if hparams.attn_residual:
            x_heads = split_heads(residual_x)  # [B, T, D] -> [B, H, T, D/H]
        if hparams.attn_proj_residual in ('approach_4', 'approach_5'):
            a, w, logits = multihead_attn(q, k, v)  # A_h V_h: [B, H, T, D/H]
            ax = merge_heads(tf.matmul(w, x_heads))  # Concat_h[A_h X_h]: [B, H, T, S] @ [B, H, S, D/H]
    ↪ -> [B, T, D]
            if hparams.attn_proj_residual == 'approach_5':
                ax = hparams.attn_alpha * residual_x + (1 - hparams.attn_alpha) * ax  # alpha X + (1 -
    ↪ alpha) Concat_h[A_h X_h]
            a = conv1d(merge_heads(a), 'c_proj', n_state) + ax  # [B, T, D]
            return a, present, w, logits
```

in `block()`

```
        a, present, w, logits = attn(norm(x, 'ln_1'), 'attn', nx, past=past, hparams=hparams,
    ↪ residual_x=x)  # norm: [B, T, D]; a: [B, T, D]; w, logits: [B, H, T, S]
```

```
        if hparams.attn_residual and hparams.attn_proj_residual != 'approach_3':
            x = a  # a = Concat_h[A_h (V_h + X_h)] W_O already carries x; [B, T, D]
        m = mlp(norm(x, 'ln_2'), 'mlp', nx*4, hparams=hparams)  # norm: [B, T, D]; m: [B, T, D]
        x = x + m  # [B, T, D]
        return x, present, w, logits
```

A.8 Complete `attn` and `block`

The full functions, unedited, with all branches.

```
def attn(x, scope, n_state, *, past, hparams, residual_x=None):
    assert x.shape.ndims == 3  # Should be [batch, sequence, features]
    assert n_state % hparams.n_head == 0
    if past is not None:
        assert past.shape.ndims == 5  # Should be [batch, 2, heads, sequence, features], where 2 is [k, v]

    def split_heads(x):
        # From [batch, sequence, features] to [batch, heads, sequence, features]
        return tf.transpose(split_states(x, hparams.n_head), [0, 2, 1, 3])  # [B, T, D] -> [B, T, H, D/H]
    ↪ -> [B, H, T, D/H]

    def merge_heads(x):
        # Reverse of split_heads
        return merge_states(tf.transpose(x, [0, 2, 1, 3]))  # [B, H, T, D/H] -> [B, T, H, D/H] -> [B, T,
    ↪ D]

    def mask_attn_weights(w):
        # w has shape [batch, heads, dst_sequence, src_sequence], where information flows from src to dst.
        _, _, nd, ns = shape_list(w)
        b = attention_mask(nd, ns, dtype=w.dtype)  # [T, S]
        b = tf.reshape(b, [1, 1, nd, ns])  # [1, 1, T, S]
        w = w*b - tf.cast(1e10, w.dtype)*(1-b)  # [B, H, T, S]
        return w

    def multihead_attn(q, k, v, x_heads=None):
        # q, k, v, x_heads have shape [batch, heads, sequence, features]
        w = tf.matmul(q, k, transpose_b=True)  # [B, H, T, D/H] @ [B, H, D/H, S] -> [B, H, T, S]
        w = w * tf.rsqrt(tf.cast(v.shape[-1].value, w.dtype))  # [B, H, T, S]

        w = mask_attn_weights(w)  # [B, H, T, S]
        logits = w                # pre-softmax attention scores (scaled, masked); [B, H, T, S]
        w = softmax(w)             # post-softmax attention weights (what the model actually uses); [B,
    ↪ H, T, S]
        if x_heads is not None:
            v = v + x_heads  # V_h + X_h: [B, H, T, D/H]
        a = tf.matmul(w, v)  # [B, H, T, S] @ [B, H, S, D/H] -> [B, H, T, D/H]
        return a, w, logits

    with tf.variable_scope(scope):
        c = conv1d(x, 'c_attn', n_state*3)  # [B, T, D] -> [B, T, 3D]
        q, k, v = map(split_heads, tf.split(c, 3, axis=2))  # three [B, T, D], each split to [B, H, T,
    ↪ D/H]
        present = tf.stack([k, v], axis=1)  # [B, 2, H, T, D/H]
        if past is not None:
            pk, pv = tf.unstack(past, axis=1)  # each [B, H, P, D/H]
            k = tf.concat([pk, k], axis=-2)  # [B, H, S, D/H]
            v = tf.concat([pv, v], axis=-2)  # [B, H, S, D/H]
        if hparams.attn_residual:
            # Head-wise attention residual: each head attends over V_h + X_h, where X_h is its
            # D/H slice of the raw (pre-ln_1) residual stream. Needs X at every source position.
            assert past is None, 'attn_residual needs the full sequence; call model() without past'
            x_heads = split_heads(residual_x)  # [B, T, D] -> [B, H, T, D/H]
        else:
            x_heads = None
        assert hparams.attn_proj_residual in (None, 'approach_1', 'approach_2', 'approach_3',
    ↪ 'approach_4', 'approach_5')
```

```
        if hparams.attn_proj_residual in ('approach_4', 'approach_5'):
            assert hparams.attn_residual, 'approach_4/5 need residual_x'
            a, w, logits = multihead_attn(q, k, v)  # A_h V_h: [B, H, T, D/H]
            ax = merge_heads(tf.matmul(w, x_heads))  # Concat_h[A_h X_h]: [B, H, T, S] @ [B, H, S, D/H]
    ↪ -> [B, T, D]
            if hparams.attn_proj_residual == 'approach_5':
                ax = hparams.attn_alpha * residual_x + (1 - hparams.attn_alpha) * ax  # alpha X + (1 -
    ↪ alpha) Concat_h[A_h X_h]
            a = conv1d(merge_heads(a), 'c_proj', n_state) + ax  # [B, T, D]
            return a, present, w, logits
        a, w, logits = multihead_attn(q, k, v, x_heads)  # a: [B, H, T, D/H]; w, logits: [B, H, T, S]
        a = merge_heads(a)  # [B, T, D]
        if hparams.attn_proj_residual == 'approach_2':
            a = norm(a, 'ln_y', affine=False)  # Y <- LN(Y); [B, T, D]
        y = a  # Y = Concat_h[A_h (V_h + X_h)], pre-projection; [B, T, D]
        a = conv1d(a, 'c_proj', n_state)  # [B, T, D]
        if hparams.attn_proj_residual in ('approach_1', 'approach_2'):
            a = a + y  # Y W_O + b_O + Y; [B, T, D]
        return a, present, w, logits


def mlp(x, scope, n_state, *, hparams):
    with tf.variable_scope(scope):
        nx = x.shape[-1].value
        h = gelu(conv1d(x, 'c_fc', n_state))  # [B, T, D] -> [B, T, 4D]
        h2 = conv1d(h, 'c_proj', nx)  # [B, T, 4D] -> [B, T, D]
        return h2


def block(x, scope, *, past, hparams):
    with tf.variable_scope(scope):
        nx = x.shape[-1].value
        a, present, w, logits = attn(norm(x, 'ln_1'), 'attn', nx, past=past, hparams=hparams,
    ↪ residual_x=x)  # norm: [B, T, D]; a: [B, T, D]; w, logits: [B, H, T, S]
        if hparams.attn_residual and hparams.attn_proj_residual != 'approach_3':
            x = a  # a = Concat_h[A_h (V_h + X_h)] W_O already carries x; [B, T, D]
        else:
            x = x + a  # [B, T, D]
        m = mlp(norm(x, 'ln_2'), 'mlp', nx*4, hparams=hparams)  # norm: [B, T, D]; m: [B, T, D]
        x = x + m  # [B, T, D]
        return x, present, w, logits
```